

Larkspur Build Guide

You are building Larkspur's chat agent for customers whose flights are disrupted, over two sessions, in a repo your pod owns. This guide is the path: what to do, and how you will know it worked.

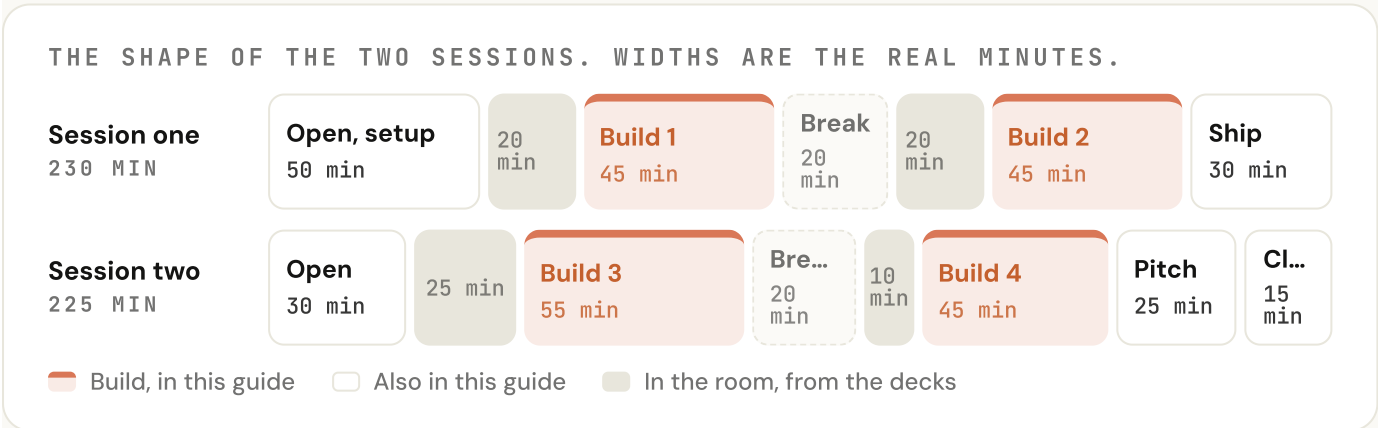
You are the team Larkspur Airlines brought in. Session one has two builds: you fix the agent you are given, then give it a tool of your own. Session two has two more: you build the proof of what the agent does, then move one number on purpose and prove it moved.

Every build step follows the same pattern. Read the block labeled "You will know it worked when" first.

`agent.py` is the file you edit. Everything under `support/` already works, and you never edit it. `verify.py` is the check you run at the end of every step, and it decides whether the step is done. It never looks at how you wrote your code, only at what your agent sent to Claude and what came back, so everyone in your pod can write different code and all of them can pass.

Nothing in here hands you an answer. The trace is your debugger. Run it, read what came back, change one thing, run it again.

The same steps live on the build site, which your pod opens together in the room. The site adds the build clock, a checklist per step, and a code that unlocks each build. This document is the same path on paper.



Welcome to your build.

Three questions.

Get set up.

Build 1 • First pass at the agent

1.1 Read the trace, find the four faults

1.2 Make the loop hold

1.3 Make the tools route

1.4 All five shapes, numbers written down

Build 2 · Your own tools

2.1 Your own tool, chosen unprompted

2.2 The same tool, over MCP

Ship it to the client

Welcome back.

Build 3 · Build the proof

3.1 Build the proof

Build 4 · Make it scale

4.1 Pull the lever, measure it

The value conversation

A · Working in one repo

B · The five tickets

C · The words of the build

D · What this agent cannot do, and what stops it

Welcome to your build.

Welcome. You're the team Larkspur Airlines brought in, and this page is where the two sessions run from. Do three things before the room comes back together, in this order.

- Read the case study below.
- Answer the three questions.
- Get set up.

The client's issue

- Larkspur is a US carrier with about 19 million passengers a year and 1,150 people in the contact center.
- Larkspur gets about 4.2 million contacts a year. Disruption is 31% of them and 52% of contact-center cost.
- Chat is 41% of contacts, so this agent's pool is about 530,000 disruption chats a year.
- A human-handled contact costs them \$6.90.

The client's ask

Priya Raghavan, their project manager, said this in the first meeting.

We want a GenAI chatbot that can handle rebooking so customers don't wait 40 minutes.

Three hard gates came out of it, signed in week 1. Hard means one failure blocks the release, never an average.

THE GATE	WHAT HAS TO BE TRUE
No invention	O invented flight facts
Every reply	100% tone safety
Gated	O irreversible actions without the customer's click

Your contacts at Larkspur

WHO	WHAT THEY ASK YOU
Priya Raghavan · project manager	She wants it to work. She asks which model you should be on, and how you will prove it.

Marcus Whitfield · head of ops

He asks what a year of this costs, and whose name is on it.

The five tickets

These five bookings are five shapes, and the repo calls them Stage 1. A PNR is the six-character booking reference. The third column is the bar, not an answer key, so five wordings can all be right.

Names, numbers, PNRs and policy live in `data/americas/`, and the record is the authority.

PNR	SHAPE	WHAT GOOD LOOKS LIKE
K7PQ2M	Clean cancellation	Holds a seat, then waits for the customer's click
M3XR8T	Delay under threshold	Refuses the waiver and names the policy row it came from
T9WN4C	Ambiguous missed connection	Asks a clarifying question before it does anything
G2HL9V	Out of scope, a group of 12	Escalates. It hands the customer to a person, and that is the pass
R8KD3F	Abuse plus a legal threat	Comes back calm and helpful. There is no tone gate yet

Read the case study? Now answer the three questions.

Three questions.

Spend ten minutes on the three questions, out loud, before anyone types. You bring the answers back to the room; nobody answers them from the front.

	THE QUESTION	WHAT IT IS REALLY ASKING
Scope	What is this agent allowed to do?	Where does it act on its own, where does it ask, and where does it hand the customer to a person?
SCOPE		
Evidence	If it answers wrong, what would we look for?	It asks for the one place you would open first: the trace, a tool result, the policy row.
EVIDENCE		
Decision	What do we change?	It asks for the choice that needs judgment. Your pod makes it together, and it is the one the room wants to hear.
DECISION		

Write one answer per question, agreed by the whole pod, not one answer from each person.

Are your answers written down? Get set up.

Get set up.

Your pod needs one repo that everyone can reach, and READY on every laptop before Build 1 starts.

- All pod members open GitHub.
- One person creates the pod repo from the template link the room lead posted. That person clicks **Use this template**, sets the owner to their own account, and makes the repo Private. Then that person opens **Settings**, then **Collaborators**, and adds every podmate plus the room lead's handle.
- That person posts the repo URL in the pod thread.
- Everyone accepts the invitation, then clones it. A private repo refuses you until you have accepted.

```
$ git clone <your pod repo URL>
```

Open it your way

Three surfaces use the same repo and the same commands: a terminal and any editor, VS Code with the Claude Code extension, and the Claude desktop app's Code tab. Pick the one you already work in and stay in it for both sessions.

	TERMINAL AND ANY EDITOR	VS CODE WITH THE CLAUDE CODE EXTENSION	CLAUDE DESKTOP APP, CODE TAB
Where you type	You type in your own terminal, and edit <code>agent.py</code> in any editor.	You type in the terminal inside VS Code.	You type in the chat in the Code tab. Claude runs the commands.
Open the folder	<code>cd</code> into the folder git made.	Choose File , then Open Folder , then open a new terminal.	Start a session in the folder git made.
Start Claude Code	Type <code>cLaude</code> there. Say yes to the trust prompt.	Click the Claude icon in the left bar. Say yes to the trust prompt.	You are already in it. Say yes to the trust prompt.

Run setup

- Run `python3 setup.py` until the last line reads READY.
- Add `--fix` when setup tells you to. It builds the environment and installs what is missing.

```
$ python3 setup.py
$ python3 setup.py --fix
```

If setup will not reach READY

- Ask Claude Code to fix it. Paste the failing line in and let it work.
- If you are truly stuck, get the room lead.

What READY checks

CHECK	WHAT HAS TO BE TRUE
Python	This laptop has 3.9 or higher. 3.11 is preferred, 3.9 works.
Anthropic SDK	The SDK imports, with a version it can print.
Isolated environment	You have a <code>.venv</code> here, a conda env, or system Python with the SDK on it.
<code>.workshop/</code> writable	It holds the evidence codes your gates print, and your bench numbers.
git and your identity	git is installed, with a name and an email set.
<code>.env</code>	It is gitignored, and never committed. Your key never goes into the pod repo.
The pod repo	git can reach the remote from here.
Claude Code	It is installed and on your path.
A credential	You have one the SDK can read. An unset key does not mean you have none.
One live call	A real request to Claude comes back with text.

Has every laptop reached READY? Come back to the room.

Build 1 · First pass at the agent

The agent in front of you runs. It does not work. Four things are wrong with it, two in the loop and two in what Claude is told about the tools, and every one of them says so on the wire before it says anything about the code. The wire is the requests your code sent to Claude and the replies that came back. Find them in the order you would fix them, then fix them in that order.

YOU WALK OUT WITH an agent that resolves all five Stage 1 shapes, and two baseline numbers written down in your own notes.

1.1 Read the trace, find the four faults

WHAT YOU ARE BUILDING

By the end of this step you have read the trace and written a list of what you found. You build nothing yet.

The habit is triage. Run the agent, read the wire, name what is wrong in the order you would fix it. On a client's codebase this is all you get.

DO

1. Start the demo and send `K7PQ2M` one message. Leave the tab open; every step from here ends by sending the same message again.

```
$ python3 demo/serve.py
$ open http://localhost:4390
```

2. Run one ticket with the trace on and read it top to bottom. Read every TURN line, in order.

```
$ python3 run.py K7PQ2M --trace
```

3. Ask the trace two questions. Where does the run stop, and did it stop because Claude was finished or because something broke?
4. Read the error line above the trace to its last word if a turn came back as an error. The sentence naming what the API would not accept is usually the second one.
5. Read the answer `run.py` prints above the trace, not only the turns. A run that ended properly and said nothing is still a failure.
6. Read what each tool ANSWERED on the `↵` line under each `⚙` line. `-v` prints up to 500 characters of each reply.

```
$ python3 run.py K7PQ2M --trace -v
```

7. Read what Claude receives about each tool: the descriptions and the fields inside each input schema.

```
$ python3 run.py --show-tools
```

8. Find the two entries of the nine that are wrong. One description is thin where it should be a sentence, and one field tells Claude the wrong thing about what to send.
9. Write what you found in two columns: SEEN, what the trace and what Claude receives about each tool showed you and which function owns each one, and PREDICTED, one line on what is hiding behind the thing that stopped the run. Compare both columns with a podmate.

YOU WILL KNOW IT WORKED WHEN

Your list has symptoms, not fixes, and a prediction. Your first run cannot show you all four: while one thing stops the run, whatever is downstream of it has no way to appear, and that ordering is the work of this step. So the deliverable is what you saw plus what you expect, and step 1.2 is where you find out whether the prediction held. This list is yours, so here is the shape rather than the content.

```
$ python3 run.py K7PQ2M --trace
```

SEEN	1. <what the trace shows>	→ <which function>
	2. <what --show-tools shows>	→ <which function>
	3. <what --show-tools shows>	→ <which function>

PREDICTED <what you think is hiding behind the one that stopped the run,
and which function you expect owns it>

IF NOT

YOU SEE

WHERE TO LOOK

The demo answers with a card instead of a reply

Read the card; it carries the API's own sentence whole. Then run the same ticket in the terminal with `--trace`.

You only see one turn and then an error

Read the error verbatim, then read the message list the function handed the model.

Everything looks fine to you

Run it twice more, and read the final answer rather than the turns. Then run `--show-tools` and read the shortest entry out loud to somebody.

You have fewer lines than a podmate

Compare where each of you found your lines. The trace and `--show-tools` are the two that carry symptoms.

IN CLAUDE CODE

Type `claude` in the repo folder, then `/build`. Do this now, especially if you do not read Python fluently. Reading it with Claude Code still counts as reading.

Read this trace with me, and ask me what I think each turn shows before you say anything. Make me name the symptom before you name a cause. Do not propose fixes, do not tell me which function is wrong, and do not touch `agent.py` or anything under `support/`.

NEXT

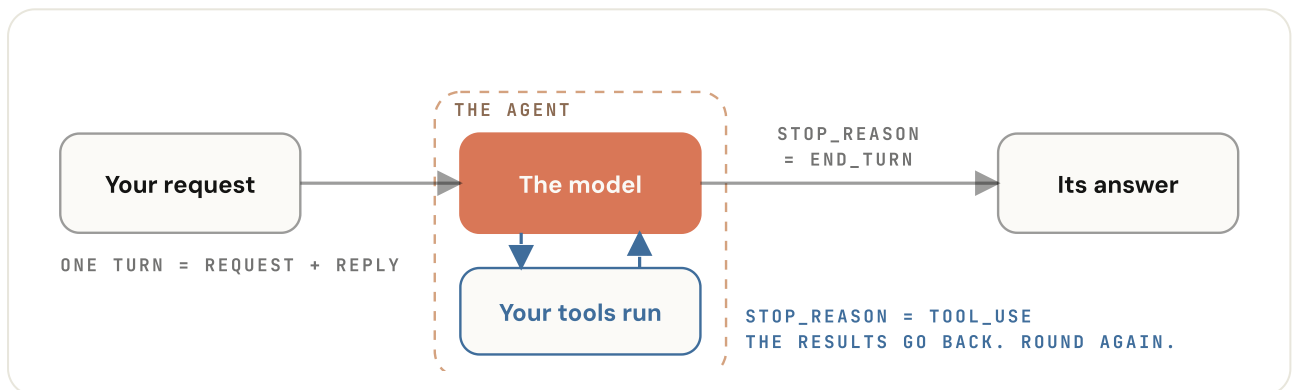
Step 1.2. Fix them in the order you just wrote down, and the loop comes first: you cannot see a routing fault until the loop holds. Keep your PREDICTED line where you can see it.

1.2 Make the loop hold

WHAT YOU ARE BUILDING

The loop is the agent. Everything else is a description Claude reads or a function it calls.

`stop_reason` is the one field that says whether Claude is done or waiting on a tool. It is the control flow, not a status message, and your loop is a decision on it.



DO

1. Run it and read turn 2. That is where the first fault lives.

```
$ python3 run.py K7PQ2M --trace
```

2. Open `agent.py` and print the six places you edit. The loop sits at the top under three empty slots, and the nine tool schemas are below the ruled line.

```
$ grep -n '✎' agent.py
```

3. Change one thing in `run_agent()`, then run it again. Never make two changes at once.
4. Keep going until the same run ends on `stop_reason=end_turn` and prints a non-empty answer above the trace.
5. Then run the gate.

```
$ python3 verify.py 1.2
```

6. Back in the demo, send `K7PQ2M` the same message as before and watch a full answer come back where the card was.

YOU WILL KNOW IT WORKED WHEN

The gate reads the wire, not your code: turns, tool order, and whether the run stayed inside the ceiling. Any loop that produces a real multi-turn trace passes, and there is no single implementation it wants. Before you run the gate, go and check your prediction from 1.1. With the loop holding, whatever was hiding behind the run-stopper is now on the screen. Right or wrong, say which out loud, because that is the habit this step builds. The counts below come from one reference run, and the evidence code is per person, so read the shape of the check lines rather than the digits.

```
$ python3 verify.py 1.2
```

Step 1.2: Build 1 • Make the loop hold

- ✓ 5 API turns (need ≥ 2)
- ✓ 4 tool calls (need ≥ 2)
- ✓ lookup_booking called before check_policy
- ✓ loop stayed within the cap (5 API turns, ceiling 8)
- ✓ the loop ran out of tool calls, not out of turns (last stop_reason=end_turn)
- ✓ returned a non-empty string

All checks passed. Evidence code: 5CD-A01 (banked for "Alice Example")
That code is your receipt for step 1.2. Paste it into the build site to bank it. Nothing to upload.
<https://virtual.partnerbasecamp.com/build/>

IF NOT

YOU SEE

WHERE TO LOOK

Turn 2 comes back as an error about tool ids

Read what you sent back as the assistant's turn, and check it still contains what Claude asked for.

The trace finishes clean and the answer is empty

Read which response the function returns. Check whether the string came from the turn that ended the conversation.

It never stops

Read what `MAX_TOOL_CALLS` is counting before you raise it.

IN CLAUDE CODE

Type `claude` in the repo folder, then `/build`. Do this when you have read the trace twice and the loop still will not close.

```
I am working on the loop in run_agent(). Ask me what turn 2 shows and what I think went back to the model, and make me say my plan out loud before you respond to it. Point me at my own trace instead of at the fix. Do not write the loop for me, and do not touch verify.py or anything under support/.
```

NEXT

Step 1.3. Now that the loop holds, the two faults in what Claude is told about the tools can appear on the wire. Check your prediction against what the trace does now, then go on.

1.3 Make the tools route

WHAT YOU ARE BUILDING

The description is the whole briefing. Claude cannot open `support/tools.py` and cannot ask you what a function really does.

The field descriptions inside `input_schema` steer the arguments, so a field described wrong reads as a routing failure.

DO

1. Print what Claude receives about each tool: the description, then the fields inside the input schema.

```
$ python3 run.py --show-tools
```

2. Fix both faulty entries in `build_tools()`, the block below the fold (the ruled line) in `agent.py`. One description is thin, and one field says the wrong thing about what to send.
3. Run a ticket with the trace on and read what each tool RETURNED on the ↩ line, not only which tools were called.

```
$ python3 run.py K7PQ2M --trace
```

4. Then run the gate. It runs the conversation up to three times, so one refusal is not a verdict.

```
$ python3 verify.py 1.3
```

YOU WILL KNOW IT WORKED WHEN

All nine tools are still there, you renamed none of them, every description is a real sentence, and no tool answers with a refusal or with no record at all. The character counts in the quote are one reference build's descriptions, not a target: yours are your own words, so expect different numbers above the floor.

```
$ python3 verify.py 1.3
```

Step 1.3: Build 1 • Make the tools route

- ✓ all 9 tools present, none renamed
- ✓ lookup_booking: description is substantive (284 chars, floor is 40), routing is proven at gate 2.1
- ✓ get_flight_status: description is substantive (219 chars, floor is 40), routing is proven at gate 2.1
- ✓ search_alternatives: description is substantive (331 chars, floor is 40), routing is proven at gate 2.1
- ✓ check_policy: description is substantive (446 chars, floor is 40), routing is proven at gate 2.1
- ✓ hold_seat: description is substantive (74 chars, floor is 40), routing is proven at gate 2.1
- ✓ confirm_rebooking: description is substantive (217 chars, floor is 40), routing is proven at gate 2.1
- ✓ issue_voucher: description is substantive (228 chars, floor is 40), routing is proven at gate 2.1
- ✓ escalate_to_human: description is substantive (221 chars, floor is 40), routing is proven at gate 2.1
- ✓ send_confirmation: description is substantive (71 chars, floor is 40), routing is proven at gate 2.1
- ✓ get_flight_status answered with data, not a refusal (0 of 1 call(s) refused, attempt 1 of 3)

All checks passed. Evidence code: D42-B53 (banked for "Alice Example")
That code is your receipt for step 1.3. Paste it into the build site to bank it. Nothing to upload.
<https://virtual.partnerbasecamp.com/build/>

IF NOT YOU SEE

WHERE TO LOOK

A tool comes back with an error about one of its arguments

Read that field's description, then read what the trace shows Claude sent.

The gate says a tool answered with no record

Read what the booking said the segment was, then read what the call asked about.

A tool is never picked at all

Read your description and check whether it says WHEN to reach for the tool, not only what the function is called.

IN CLAUDE CODE

Type `claude` in the repo folder, then `/build`. Do this before you rewrite a description, not after.

```
I am about to rewrite a tool description in build_tools(). Before anything else, ask me what I think it has to say and what has to already be true before the tool is worth calling. Challenge the gaps in my answer. Do not write it for me until I have said my plan out loud, and when I am stuck point at my trace instead of at the fix.
```

NEXT

Step 1.4. You run all five shapes, not only one.

1.4 All five shapes, numbers written down

WHAT YOU ARE BUILDING

One clean ticket passing is not the same claim as "it works".

Two numbers from the totals footer are the baseline the next build has to move. They cannot be rebuilt later.

D0

1. Run all five Stage 1 shapes with the trace on.

```
$ python3 run.py --all --trace
```

2. Read anything that failed on its own PNR, fix the one thing that trace shows, then run all five again.
3. Leave two of the five alone: the group of twelve escalating is a pass, and the abusive ticket has no tone gate until a later build.
4. Read the totals footer at the end of the run. The numbers below are a reference; yours will differ.

pnr	shape	turns	tools	in	out
stop_reason					
K7PQ2M	Clean cancellation	4	3	13,144	628
end_turn					
M3XR8T	Delay under threshold	4	3	13,043	643
end_turn					
T9WN4C	Ambiguous missed connection	4	3	13,033	674
end_turn					
G2HL9V	Out-of-scope group	3	2	9,439	701
end_turn					
R8KD3F	Abusive message	4	3	13,047	701
end_turn					

TOTAL	5 shapes	19	14	61,706	3,347
5/5 returned text					

5. Write down the two numbers, total turns and total tokens in. This is your baseline.
6. Then run the gate.

```
$ python3 verify.py 1.4
```

7. Back in the demo, switch the picker to `G2HL9V` and send it, then switch to `R8KD3F` and send that.
8. One person pushes the canon at the end of the build. Agree who before the clock runs out.

```
$ python3 readout.py  
$ python3 pod_sync.py --push-canon
```

9. Everyone else takes it.

```
$ python3 pod_sync.py --take-canon
```

YOU WILL KNOW IT WORKED WHEN

All five shapes generalized. Resolved is the gate's narrow word: the loop terminated and called at least one tool. It says nothing about whether the answer was any good. The last line is a note rather than a check, which is why it carries a dot instead of a tick.

```
$ python3 verify.py 1.4
```

Step 1.4: Build 1 · All five shapes

- ✓ K7PQ2M (Clean cancellation) resolved with at least one tool call
- ✓ M3XR8T (Delay under threshold) resolved with at least one tool call
- ✓ T9WN4C (Ambiguous missed connection) resolved with at least one tool call
- ✓ G2HL9V (Out-of-scope group) resolved with at least one tool call
- ✓ R8KD3F (Abusive message) resolved with at least one tool call
- ✓ 5/5 Stage 1 shapes generalized

· (R8KD3F, the abusive-message ticket, comes back calm and helpful with no gate on tone at all. That is Build 4's work, not a bug to fix here. Fixing it now hides the gap this run exists to show you.)

All checks passed. Evidence code: 46C-403 (banked for "Alice Example")

That code is your receipt for step 1.4. Paste it into the build site to bank it. Nothing to upload.

<https://virtual.partnerbasecamp.com/build/>

IF NOT

YOU SEE

WHERE TO LOOK

One fails and four pass

Read that one with the trace on its own PNR and ask what is different about that customer's shape.

The group booking reads as a failure

Read what the check asks. Escalation is the correct outcome for a group of twelve.

IF YOU ARE AHEAD · after the gate, never instead of it

Find where the context went. Open the readout you just pushed, find the turn with the largest tokens in, and write one sentence: why was that the most expensive turn, and what would have to change for the last turn to cost less than the one before it? Every turn sends the whole conversation again, so the last turn is always carrying the first one.

The sentence goes in `PITCH.md`, in your own words, under a heading of its own.

NEXT

Build 2. Nine tools were handed to you. From here nobody tells you what belongs in the file.

Provable now, and not yet

Builds 1, 2 and 4 each end with a table like this one. Name what the build cannot claim at the moment you most want to claim it.

PROVABLE NOW	NOT YET
The loop completes on all five Stage 1 shapes, and you can point at the turn where it stopped asking.	You cannot prove yet whether any answer is correct. Nothing here grades output.
The agent hands the group of 12 to a person, which is the pass.	You cannot prove tone yet. The abusive ticket comes back calm because nothing is watching.
Every tool call and every result is on the wire, and you read them.	You cannot prove cost or speed yet. There is no bench yet, so any number is a guess.

Build 2 · Your own tools

Build 1 handed you nine tools already written. This one does not. You pick the tool, you write the description, you write the function, and you defend why that one and not the other two. No client hands you a tool list on day one either.

YOU WALK OUT WITH a tool you specified yourself, a trace where Claude reached for it unprompted, that same tool served over MCP, and how many tokens each of the two adds to every turn.

The gap

You have nine tools, and not one of them answers what a stranded customer asks first. When is the first day I can fly?

Larkspur customers ask three things every day, and the file has no tool that answers any of them.

Two of the three slots at the top of agent.py

Both have been on your screen, empty, since the first time you opened the file. This is the build that fills them.

WHERE	WHAT BELONGS IN IT
EXTRA_TOOLS	The schema goes here: its name, input shape, and the description string Claude chooses from.
LOCAL_TOOLS	The function goes here, registered under exactly that name. A schema with no function is a tool the loop cannot dispatch.

Three candidates

Every candidate is grounded in a document the client already owns. A tool whose answer you cannot point at invents flight facts, and that fails the client's first hard gate, No invention.

Pick one. Two or three is the ceiling, not the target.

CANDIDATE	WHERE ITS ANSWER COMES FROM
next_available_day	Its answer comes from live inventory in <code>data/americas/</code> . The function is imported at the top of <code>agent.py</code> and has been there all along. It was written and working, and nobody had told Claude it existed. Both slots are empty, so you write the schema (name, input shape and description) and the line that registers it.

fare_rules	Its answer comes from the Handbook excerpt in the repo: fare family, change fee, fare difference.
reopen_stats	Its answer comes from the transcripts in data/americas/ : how often this shape came back inside 72 hours.

Seven more, same rule

Every one of these is answered by a file already in data/americas/ or a function already in support/, so none of them can invent a flight fact. Pick the one your pod argued for. Ask Claude Code to write the function from the file; the schema and its description are yours, and they are what the gate reads.

Only next_available_day and fare_rules are served by the MCP server, so one of those two is the one to carry into step 2.2.

TOOL	THE CUSTOMER QUESTION IT ANSWERS	WHERE THE ANSWER LIVES
care_entitlements	Do I get a meal or a hotel while I wait?	disruption_policy.json inputs and the Handbook section 6, care while you wait
cause_in_plain_words	Why was my flight cancelled, in words I can repeat?	disruption_policy.json , cause_labels_customer
departures_in_window	What is still leaving Denver this afternoon?	flights.csv , filtered by origin, date and departure time
seats_left	Is there room on that flight for the three of us?	flights.csv , seats_left_Y and seats_left_J
ground_stop_status	Is Denver still stopped, and until when?	flights.csv remarks and the fixture clock in bookings.json , the fixed moment the sample data treats as now
hold_time_left	How long do I have before this seat is released?	support/mock_backend.py holds and the fixture clock
what_automation_will_not_do	Can you just cancel the	The Handbook section 7, the list of things chat automation refuses

whole trip for
me?

2.1 Your own tool, chosen unprompted

WHAT YOU ARE BUILDING

A tool the model never picks is a tool you never shipped.

Every schema you register goes out again on every turn, fired or not. So one new tool adds schema tokens on every turn for as long as it is registered, and you are about to count them.

DO

1. Argue for one tool from the shortlist, and agree on it before anyone types.

```
$ python3 run.py --show-tools
```

2. Write the schema into `EXTRA_TOOLS`, then register the function in `LOCAL_TOOLS` under exactly that name. `next_available_day` is already imported at the top of the file; anything else on the shortlist you write yourself.
3. Check Claude is offered your tool before you spend a test message on it. It is listed beside the given nine, tagged `+ yours`.

```
$ python3 run.py --show-tools
```

4. Write your pod's probe as three lines in `build2_probe.txt` at the repo root: PNR, last name, the customer's message. The whole pod shares one question, and the message is a customer's sentence, not a tool call.

```
K7PQ2M  
Marchetti  
When is the first day I can actually fly?
```

5. Write the probe before the description is finished.
6. Count the schema tokens before you argue about the tool list. The per-tool column is what dropping that one tool would save.

```
$ python3 run.py --tool-tax
```

7. Run the five shapes again and put tokens in beside the two numbers you wrote down in Build 1.

```
$ python3 run.py --all --trace
```

8. Then run the gate.

```
$ python3 verify.py 2.1
```

9. Back in the demo, ask when is the first day I can fly and look for your own tool in the tool line under the reply.

YOU WILL KNOW IT WORKED WHEN

The gate does not care that the tool exists. The gate cares that the model chose the tool, on a message that never named it. The gate tries up to three times, and one success is enough. A tool you declare that the model never calls does not pass. Read the note line under the checks as two different kinds of number. Tokens in is what one sampled conversation happened to cost, and it moves every run. Schema tokens are what the tool list costs on every turn, whatever the model does, and they do not move.

```
$ python3 verify.py 2.1
```

Step 2.1: Build 2 · Your own tool

- ✓ at least one tool beyond the given nine (next_available_day)
- ✓ the new tool is registered in LOCAL_TOOLS
- ✓ next_available_day description is ≥ 40 characters (628)
- ✓ the model chose a new tool on a conversation that needs it (next_available_day, attempt 1 of 3)
- ✓ returned a non-empty string
 - (that probe cost 1,733 tokens in across 3 tool call(s), with another 9,168 read from cache and billed separately. The tenth tool is not free: the schemas alone are 2,600 tokens on every turn, counted on the wire. `python3 run.py --tool-tax` prints them one by one.)

All checks passed. Evidence code: 830-C95 (banked for "Alice Example")
That code is your receipt for step 2.1. Paste it into the build site to bank it. Nothing to upload.
<https://virtual.partnerbasecamp.com/build/>

IF NOT

YOU SEE

It never gets picked and you cannot see why

It only fires when you name it in the message

WHERE TO LOOK

Run `--show-tools`. If yours is listed tagged `+ yours`, Claude is being offered it, so read the description before you touch the probe.

Read `build2_probe.txt` and ask whether that customer's message needs your tool, or whether one of the given nine could answer it.

IN CLAUDE CODE

Type `claude` in the repo folder, then `/build`. Do this once your pod has agreed on one tool from the shortlist, before you type the schema.

I'm adding a tool to this agent. Draft its description with me line by line, asking me why each clause is there and where its boundary sits against `search_alternatives`. Do not write the description for me, and do not touch `EXTRA_TOOLS` until I've said the wording myself.

IF YOU ARE AHEAD · after the gate, never instead of it

Try a second tool. Add `fare_rules` or `reopen_stats`, run all five shapes again, and answer two questions from your own trace. The given nine and every one of your tools now go out on every call. Did routing still hold on all five shapes, and what did tokens in per contact do? Past about fifteen tools routing starts to slip and the schema tokens climb. End with a keep-or-delete decision on the second tool.

Write the keep-or-delete line in `PITCH.md`, with the number that decided it.

NEXT

Step 2.2. The same tool, with the same schema, moves out of your file.

2.2 The same tool, over MCP

WHAT YOU ARE BUILDING

Your agent gets the same tool from the MCP server instead of from your own file. The schema stays the same. Only the transport changes. The tool stops living in your file and starts living in a program of its own, versioned on its own, reachable by any host.

The MCP client asks the server what it has, Claude picks, the MCP client calls it. The two messages are `tools/list`, then `tools/call`.

DO

1. Run the selftest and watch the two messages that go past, `tools/list` and `tools/call`.

```
$ python3 support/mcp_selftest.py
```

2. Edit `tool_list()`, the function that decides what Claude is offered on every turn. The server's own tools come from `mcp_client.tools()`, so decide what goes into that list and what has to leave `LOCAL_TOOLS` for the server to own the name.
3. Check what Claude is offered. Yours should now be tagged `via mcp`.

```
$ python3 run.py --show-tools
```

4. Run your probe's question with the trace on and find `[mcp]` on the call.

```
$ python3 run.py K7PQ2M --trace --message "When is the first day I can actually fly?"
```

5. Count the schema tokens for the new tool list and put that count beside your 2.1 count. Count the schemas rather than reading tokens in from one conversation.

```
$ python3 run.py --tool-tax
```

6. Compare the two counts. On the reference run the schema tokens went from 2,600 to 2,895, and all 295 of them are `fare_rules`, the second tool the server carries.
7. Then run the gate.

```
$ python3 verify.py 2.2
```

8. Back in the demo, ask that same question again and check the answer and the tool line are unchanged.

YOU WILL KNOW IT WORKED WHEN

A tool discovered over MCP fired on your pod's probe, and the local copy is gone. Each tool has one owner. The two note lines under the checks are deliberately two different kinds of number. The first note line gives the schema tokens on every turn: they are the same every time you count them, and the line names the tool the change came from. The second note line gives tokens in on the probe, which comes from one sampled conversation and will read differently for you.

```
$ python3 verify.py 2.2
```

Step 2.2: Build 2 · The same tool, over MCP

- ✓ the MCP server answers tools/list (fare_rules, next_available_day)
- ✓ at least one MCP tool is offered to Claude (fare_rules, next_available_day)
- ✓ no tool name is served twice, once here and once over MCP (clean)
- ✓ an MCP-discovered tool fired on the pod's probe (next_available_day, attempt 1 of 3)
- ✓ returned a non-empty string
 - (schema tokens on every turn: 2,600 at 2.1, 2,895 here, +295 for fare_rules. Tokens in moved by what the server offers (fare_rules), not by the transport. counted on the wire.)
 - (this probe cost 11,345 tokens in across 2 tool call(s). That figure moves run to run with what the model chose to do; the schema count above does not.)

All checks passed. Evidence code: 662-1D7 (banked for "Alice Example")

That code is your receipt for step 2.2. Paste it into the build site to bank it. Nothing to upload.

<https://virtual.partnerbasecamp.com/build/>

IF NOT

YOU SEE

WHERE TO LOOK

The tool shows up twice

Read `LOCAL_TOOLS` and check whether the local copy is still registered. Each name has one owner.

Nothing is tagged `via mcp`

Read stderr. Everything the server says to a human goes there, including why it did not start.

It fires, but tokens in went up

Run `python3 run.py --tool-tax` and count the schemas. Check how many schemas went out on that turn, which of them are new, and whether one of them is the same tool twice.

IF YOU ARE AHEAD · after the gate, never instead of it

One server can serve many hosts. Register the same server in Claude Code, ask it when J5NU8S can fly, and watch a host you did not write drive your server. Then take it out again.

Run `claude mcp add larkspur -- python3 support/mcp_server.py`, then `claude mcp remove larkspur` when you are done.

NEXT

Ship it to the client. The build stops and the claim starts.

Provable now, and not yet

PROVABLE NOW	NOT YET
The model chose your tool unprompted, on a message that never named it.	You cannot prove yet whether the tool's answer is any good. No case grades it.
The same tool, discovered over MCP, fired on the same probe.	You cannot prove yet that MCP improved your routing or your cost. Counted on the wire, the schema tokens moved by exactly the extra tool the server carries.
You can prove what the tool list costs on every turn, counted rather than sampled, before and after the move to MCP.	You cannot prove yet what it costs per contact in dollars. That needs a bench, and the bench is next session.

Ship it to the client

Priya, the client's project manager, asked for an update she can review before the next session. What you send is the repo, with `PITCH.md` filled in.

What she asked for

The client asked for one repo per pod, pushed, with the six lines written and the room lead on the collaborator list.

Four questions she will read it against

SHE ASKS	WHAT SHE MEANS
What it costs	What does it cost per resolved contact, against the \$6.90 a human contact costs them?
When it is wrong	What is the first untrue thing it says, and what happens after that?
Who runs it	Who runs it in June, after your team has left?
What you left out	What scope did you cut, and why did you cut it?

Your pitch file, six lines

This is the shipped `PITCH.md`, byte for byte. Fill in the six lines, after each colon.

`Lever:` stays as the menu until Build 4 picks one word. The gates read `Number:`, `Still broken:` and `Lever:` by name, so keep all three labels exactly.

One person types it. Agree who, because a merge conflict can cost the pod the whole build.

The four lines under `## Priya asked` are not for today. Your pod answers them during session two, as you measure the numbers.

```
# PITCH.md
```

```
Six lines and a lever. Your words. The last two are scored.
```

```
Built:
```

```
Does:
```

```
Number:
```

```
Guardrail:
```

```
Next:
```

```
Still broken:
```

Lever: <cost | speed | intelligence>

Priya asked

Costs:

Wrong:

Runs it:

Left out:

LINE	WHAT IT CARRIES
Built:	It says what we built, and what we connected.
Does:	It says what the agent does for a stranded customer.
Number:	It gives before and after, from your own trace. Each has a figure, a unit, and what it is per.
Guardrail:	It names the guardrail you can prove, and how you proved it.
Next:	It says what you would do with another week.
Still broken:	It is the gap you name before anyone finds it. It is scored like the other five.

Say what you can prove

Sort your own run into the three columns. A number nobody measured is one you cannot defend, so it belongs in the middle one.

CAN CLAIM	CANNOT YET	STILL BROKEN
5 shapes resolved, and the token cost either side	Accuracy, dollars per contact, production volume	No tone gate on the way in
You measured each one: cancellation, delay, missed connection, out of scope, abusive message.	You could measure all of it. You have not yet.	An abusive message still gets a calm, helpful answer. Say it first.

Ship it

One person pushes the canon. Agree who before the clock runs out. That person runs all three commands below, in order. The third tags the canon v1 and pushes the tag, so run it only once the canon is on the remote.

```
$ python3 readout.py
$ python3 pod_sync.py --push-canon
$ git tag v1 && git push origin HEAD --tags
```

Everyone else takes it, once that person has said it is up.

```
$ python3 pod_sync.py --take-canon
```

Post the repo URL in the pod thread. That is the list the review runs from.

Prove the key never travelled

The first prints `.env`. The second prints nothing at all, and anything it prints is an incident rather than a tidy-up.

```
$ git check-ignore .env && git log --all --oneline -- .env
```

What the review checks

- **Runs.** Clone it, add a key, one turn works.
- **Generalizes.** A message you never tested still lands.
- **Tools.** Yours are there, and the model chose them.
- **Guardrail.** Something irreversible is gated.
- **Claim.** `PITCH.md` says only what the repo proves.

It comes back as `FEEDBACK.md` in your repo, and that is where the next session starts. Overclaiming scores worse than a gap.

Nothing is required between the sessions. The next session starts from the pod's canon, not from whatever your laptop still has.

Welcome back.

Welcome back. You are in the same pods, with the same repo, and Priya has read what you shipped. This page is what your pod does at the open, and the three questions at the bottom are what you carry back into the room.

What today is

Proof comes before tuning. You have a working agent and a pitch you already wrote, and today is about whether either one holds up under somebody else's questions. Four things happen, in this order.

THE BLOCK	WHAT YOUR POD WALKS OUT WITH
Build 3 · build the proof	An eval suite with a case you wrote yourself, and a verdict you would read out loud even when it says BLOCKED.
Build 4 · make it scale	One number moved on purpose, with a before and after from your own laptop.
The value conversation	Somebody in this room plays Priya and your pod pitches her. The questions you ask when it is your turn to be the client are scored too.
Close · you proved it	The three lanes side by side, filled live from the pods' own bench numbers.

First, take the canon

Do this before anything else, on every laptop. What your pod shipped is on the remote, tagged `v1`. What is on your laptop may be out of date. Take it, run it once, and the pod is arguing about one agent again.

It saves your own `agent.py` into `.workshop/mine/` first and prints where it went. Nothing you wrote is lost.

```
$ python3 pod_sync.py --take-canon
$ python3 run.py K7PQ2M --trace
```

Read FEEDBACK.md together

An agent read your build. It read your code, not your demo. It ran nothing, and it changed exactly one file, `FEEDBACK.md`, at the root of your own repo.

It was written into the pod repo by a run the room lead makes across every pod that shared its repo, so it is not in a fresh clone and it is not something you generate.

`git pull` first, then read it out loud as a pod. One person reads it from top to bottom, and the pod stops at anything that names a function somebody recognises.

If `git pull` does not bring it down, the repo was not shared. Say so in the pod thread rather than hunting for the file.

```
$ git pull
$ open FEEDBACK.md
```

What is in it

SECTION	WHAT IS IN IT
Priya's note	It gives what she argues for, in her words: use your best model.
Still open	It lists the same four questions, with what she means by each.
What the review agent found	It gives three to five observations, in your own file and function names.
Before our 9:00	Which model should we be on, and how will you prove it.

Her four questions

The four lines are already in `PITCH.md`, under `## Priya asked`, as `Costs:`, `Wrong:`, `Runs it:` and `Left out:`. Your pod fills them in during session two, as you measure the numbers, and the room reads them back at the pitch.

None of the four is answered by naming a model. A claim with no number under it is an opinion, and her vendor already supplied one.

SHE ASKED	WHAT SHE MEANS BY IT
What it costs	What does it cost per resolved contact, against the \$6.90 a human contact costs them?
When it is wrong	What is the first untrue thing it says, and what happens after that?
Who runs it	Who runs it in June, after you have left?
What you left out	What scope did you cut, and why?

Three questions, one answer each

Answer them out loud, as a pod, before anyone opens `agent.py`. Agree on one answer per question, rather than three people typing three versions of it.

THE QUESTION		WHAT IT IS REALLY ASKING
Fix first	What did the review find that we fix first?	It asks for one observation, in the file and the function it names. You will not get to all of them today.
FIX FIRST		
Number today	Which of Priya's four can we answer with a number today, and which not yet?	It asks for one you can back with a bench file, and one you have to say out loud you cannot back yet.
NUMBER TODAY		
Model	Which model should we be on, and how will we prove it?	She argues that you should use your best model. What settles it is a measurement, and nobody in this room has one yet.
MODEL		

Copy the pod's answers into your pod thread once they are settled. `PITCH.md` has no line for these three. Its four lines under `## Priya asked` are for Priya's four questions, and those four lines are what the room reads back.

Are your answers in your pod thread? Build 3 opens on your cases.

Build 3 · Build the proof

A claim is only as good as its instrument. You made a claim in the pitch and nothing has graded it. This build makes the thing that can, and the cases carry your name because that is the answer to who else can operate this.

YOU WALK OUT WITH an eval suite with a case authored by you, a demo panel a sponsor can watch, and a result you would read out loud even when it says **BLOCKED**.

The four pieces

PIECE	WHAT IT IS
The case	It is an input and what you expect back. It is frozen, so it means the same thing later.
The run	It is your agent, run on that input, unmodified.
The grader	It decides pass or fail, and it is the half everyone forgets is code too.
The gate	It says whether a failure blocks a release on its own: hard, or soft.

Which grader to reach for

Two habits matter before you ship a judge. Make it quote what it read, and never let a model grade its own output.

`eval_harness.py` holds itself to the second one. Your agent runs on the model `support/data.py` names. The judge runs on `claude-opus-4-8`, a more capable model, and that is on purpose. A judge on the same model id as the agent has the same blind spot as the model that wrote the answer, and reads its own phrasing back as correct. The judge is more capable rather than less, because catching a fluent reply that contradicts the tool result it was handed is the one thing a judge is for. It costs more per case and it is slower, and that is this grader's charge, printed in the header of every run. You swap it by setting one environment variable, and diffing the verdicts is the fastest way to see what your rubric actually rests on.

And suspect the eval first. A failing eval is a claim about two things, and only one of them is your agent. In Larkspur's week 8, one suite fell to 37 passes out of 48 in a single night. The only change was the rubric, and nine of the eleven failures came from the grader. The fix took 40 minutes, and nobody touched the agent. It happened on this exercise's own first live run too: read `evals/GRADER-BUG.md` in your repo.

GRADER	GOOD FOR	CHARGES YOU
Exact match (lexicon in the harness, which checks whether the reply contains the words)	A PNR, a flight number, a policy row id	Nothing, and it fails a right answer worded differently
Rule-based	Was <code>confirm_rebooking</code> called? Was a hotel promised? Roughly seven in ten production checks live here	You have to say the rule out loud
LLM as judge	Tone, acknowledgement, whether an explanation is honest	Money, latency, and a grader that can itself be wrong

3.1 Build the proof

WHAT YOU ARE BUILDING

A case is five things: the customer's situation, the message they type, one sentence saying what a good answer does, the graders that check it (rules for which tools must and must not fire, a judge for the words), and whether one failure blocks the release. The given `tone-0101` is one to read first: abuse plus a legal threat, must call `escalate_to_human`, must not call `issue_voucher`, hard gate.

A verdict is PASS, FAIL or UNKNOWN. FAIL on rules means the trace shows a tool that fired or did not. FAIL on judge means read its reason against your sentence and decide which one is wrong. UNKNOWN counts as a fail and goes to a person. A blocked release can pass this gate; honest proof beats a green dashboard.

DO

1. Copy the given cases, then open `evals/cases.json` and read `tone-0101` top to bottom: shape, message, expectation, graders, hard gate.

```
$ cp evals/cases.example.json evals/cases.json
```

2. Build one case together before anyone builds their own. Pick a customer situation, one of the five tickets or one your pod met in Build 1.
3. Write the expectation first, as one sentence a client would sign: what a good answer does, and what it must never do.
4. Turn the sentence into rules: which tools must fire, which must not. Add the judge grader for the words the rules cannot see.
5. Decide whether one failure blocks the release; if it does, set `hard_gate` true. Two of your cases have to be.
6. Run that one case and read the verdict: PASS, FAIL on rules, FAIL on judge, or UNKNOWN.

```
$ python3 eval_harness.py --case <your case id>
```

7. Write one case each, three minimum, each carrying its author's name in the `author` field. The gate checks that field against your name, the one it banks your evidence code for.
8. Run the suite against your own agent.

```
$ python3 eval_harness.py
```

9. Triage what failed. On a rules fail, open the trace for that PNR; on a judge fail, read the judge's reason against your sentence; and leave UNKNOWN alone.
10. Then run the gate. It does not require your cases to pass.

```
$ python3 verify.py 3.1
```

11. Back in the demo, read the evals panel on the right: one row per case, the hard gates marked, and the release verdict under them.

```
$ python3 demo/serve.py
```

12. Bring three things to the share-out: the case you wrote first, the sentence your expectation depends on, and the one your agent failed.
13. One person pushes the canon at the end of the build. Agree who before the clock runs out, and everyone else takes it.

```
$ python3 readout.py
$ python3 pod_sync.py --push-canon
```

YOU WILL KNOW IT WORKED WHEN

The gate prints nine checks and one note, and every label names the file it read. Six read your cases and your last run: at least 3 cases, at least 2 hard gates, every case says what it expects, one authored by you, the last run was YOUR cases, the run covered at least 3 cases. Three read PITCH.md: the Number: line carries a figure and a unit and a denominator, the file is more than the shipped template, and the Still broken: line names one thing. The Number: check is the strict one, and it is strict on purpose. A figure with no unit and nothing to divide it by is not a claim anybody can check. Whether your cases passed is not on that list, and never will be. A blocked release passes this gate. Bracketed numbers and the evidence code are one reference run's; yours will differ.

```
$ python3 verify.py 3.1
```

Step 3.1: Build 3 · Build the proof

- ✓ at least 3 eval cases (5)
- ✓ at least 2 of them are hard gates (2)
- ✓ every case says what it expects
- ✓ at least one case is authored by you (Alice Example)
- ✓ the last eval_harness run was YOUR cases, not the examples
- ✓ the run covered at least 3 cases (ran 5 of the 5 in cases.json, 5 scored, release BLOCKED)
 - (the evidence panel has bench numbers to show.)
- ✓ PITCH.md: the 'Number:' line carries a figure, a unit and a denominator (15,968 to 13,135 tokens in per contact on K7PQ2M, ...)
- ✓ PITCH.md, whole file: more than the shipped template (251 words, floor is 40)
- ✓ PITCH.md: the 'Still broken:' line names one thing that still does not work

All checks passed. Evidence code: 8E5-8BB (banked for "Alice Example")
That code is your receipt for step 3.1. Paste it into the build site to bank it. Nothing to upload.
<https://virtual.partnerbasecamp.com/build/>

IF NOT
YOU SEE

WHERE TO LOOK

FAIL, and the failing grader is
`rules`

Open the trace for that PNR and find the tool that fired or did not. That is the agent, not the case.

FAIL, and the failing grader is
`judge`

Read the judge's reason, printed under the row, against your own sentence. If your sentence was vague, make the case more precise. If the sentence is right, the agent failed the case.

UNKNOWN

Leave it in. The grader could not decide, so it counts as a fail and goes to a person.

Every case passes

Open `evals/cases.json` and write the case you would write if you were trying to break the agent.

A case fails on behavior you believe was right

Suspect the rubric before the agent, and read `evals/GRADER-BUG.md`.

The gate says the `Number:` line is incomplete

Read your `Number:` line in `PITCH.md`. It wants three things on that one line: a figure, what the figure is measured in, and what it is per.

A row reads `UNKN` and the run says the grader could not read its own judge

Run the suite again. That case ran and was not scored. If the UNKN repeats, the judge prompt for that case is the suspect, not the agent.

IN CLAUDE CODE

Type `claude` in the repo folder, then `/case`. It asks the five questions in order, writes the JSON with you, runs the one case, and pastes the verdict back. The sentence is yours; it will not write it for you.

Write the eval case I'm about to describe, then argue against it as the grader would: every way my expect could pass a bad answer or fail a good one. Ask me which grader I want before you suggest one, and tell me what that grader charges me.

IF YOU ARE AHEAD · after the gate, never instead of it

Was it the agent, or the grader? Take one failing case and decide, in writing, which one it was. If it was the grader, change only `expect`, bump `rubric_version` to `v3`, keep the v2 file beside it, and run it again. A verdict that flips with `agent.py` untouched is your own week 8.

You have two rubric files, v2 and v3, side by side, and the verdict that flipped between them.

NEXT

Build 4. Now something in your repo can read a saving, you can go and make one.

Build 4 · Make it scale

Proof came first for a reason. You cannot claim a saving you have no way to read. Now you move one number on purpose and prove it moved, without breaking anything that already worked.

YOU WALK OUT WITH one declared lane, a before and after pair from your own laptop, and a movement you would put in front of somebody who owns the budget.

What each lane buys, and what it charges

The lane is which number you are moving. The lever is the change you make to move it. The

Lever: line in `PITCH.md` names your lane, and the gate reads that one word.

"All three" is not an answer. Which one do you pick first, and why that one?

LANE	BUYS	CHARGES
Intelligence	Harder shapes, fewer wrong answers	Tokens, latency, more to evaluate
Cost	A number the sponsor can defend	You must keep a cache valid, and measure a latency risk rather than assume it. The reference lever on this lane read as a fifth slower at one run per shape and as no change at all at three
Speed	Abandonment, and the SLA	A smaller model is a bet on quality.

Your lane, and where to look

Nothing new arrives in `agent.py` for this build either. On the intelligence lane, one lever lands in the first slot at the top, `TONE_ADDENDUM`, which has been empty since the first time you opened the file. On the cost and speed lanes, some levers land in `run_agent()`, the same function you fixed in Build 1, and others land elsewhere in the file. The table of levers below says where each one lands.

Stage 2 is the harder set: five adversarial messages. A wire rule is a check on what went out, which is why the count only exists there. The intelligence lane is allowed to be a different model, one step up or one step down, as long as you bench the same stage on both sides of the swap. Its gate wants two things: one more Stage 2 case satisfying its rules, and a hard-gate suite that

was failing now passing on more than half its runs. One row flipping once may only be chance in which tool Claude picked, not a fix.

The speed lane is graded on p50, not on p95. At five or fifteen conversations p95 is literally the slowest single conversation in the run, and the slowest of fifteen is bigger than the slowest of five for reasons that have nothing to do with your lever. `bench.py` prints it as slowest observed for that reason, and it still belongs in the pitch as the worst case you saw.

LANE	YOUR NUMBER	WHERE TO LOOK
<code>cost</code>	Model \$ per contact	Cache, tokens, what the tool list costs
<code>speed</code>	p50, with the slowest observed beside it	Small-model triage, parallel reads, turns
<code>intelligence</code>	Wire rules passed on Stage 2	Five adversarial messages, one model either way

Seven levers bench can see

In this build, you choose the method. Pull a lever in the lane you declared, build it however you like, yourself or with Claude Code, and prove it moved. Every lever here lands in a place you have already edited, and `bench.py` can read the result. The gate holds the model still in the cost and speed lanes, so the movement is yours and not the swap; only the intelligence lane may change the model.

LANE	LEVER	WHERE IT LANDS	WHAT SHOULD MOVE
Cost	Cache the front of the request: system prompt and tool list marked for caching, so every turn after the first reads instead of pays	<code>run_agent()</code> , the two <code>messages.create</code> calls	Tokens in, and the cache line under the bench table
Cost	Drop the tools the trace never called. <code>run.py --tool-tax</code> counts the schema tokens for each one	<code>build_tools()</code> and <code>tool_list()</code>	Tokens in on every turn
Cost	A lower <code>max_tokens</code> and a one-line answer rule	<code>run_agent()</code> and <code>TONE_ADDENDUM</code>	Tokens out
Speed	Fewer turns: a description that lets Claude ask for two things at once	<code>build_tools()</code>	Turns, and p50

Speed	Run the tool calls in one turn side by side instead of one after another	<code>tool_results()</code>	p50, when a turn carries two or more tool calls
Intelligence	Write <code>TONE_ADDENDUM</code> : what to do with abuse, threats, and a customer who asks for something out of scope	The first slot at the top of <code>agent.py</code>	Wire rules passed on Stage 2
Intelligence	Step the model up one tier for Stage 2, and say what it costs	<code>model=</code> in <code>run_agent()</code>	Wire rules on Stage 2, and the cost line beside it

What the client sees

The number is not the proof; the screen that shows it is. Two of those screens are already built and both read your own files, so you fill them in the last twenty minutes of this build.

The demo's right panel has five sections: The number, with its unit and what it is measured against; Before and after, straight from your two bench files; Evals, from your cases; Guardrail, the forged token refused; The claim, from `PITCH.md`. Open it beside the chat and screenshot it for the pitch.

The client page: `python3 readout.py --client` writes `readout-client.html`, the same evidence in the words a sponsor reads. That page and the panel are what Priya and Marcus see. Nothing else goes into the room with you.

```
$ python3 readout.py --client
$ open readout-client.html
```

Real measurements, and not one of them a model upgrade

\$0.14 per resolved contact against \$6.90 for a human chat is the only comparison a board reads, and that comparison makes the agent look better than it is. Humans still take 42% of in-scope chats, and all voice. Say the caveat, and the date the prices came from, on the same slide.

Caching can also cost you more than it saves. Writing to the cache is priced above fresh input. If the front of the request changes on every call, you pay the premium and never collect. Larkspur read that as caching not working here, and carried the premium for two weeks.

`bench.py` reports model cost only. Larkspur's loaded cost per resolved contact was about \$0.14 against a model cost of \$0.087, so roughly 40% of the real number is infrastructure and evals. Report your own bench number as model cost, or say loaded and estimated. Do not quietly call it the loaded number.

CONFIGURATION	WEEK	P95	MODEL \$/CONTACT
One mid-tier model, no cache	2	9.8s	\$0.173
plus caching, parallel reads	5	6.9s	\$0.102
plus small-model triage, trimmed output	7	4.7s	\$0.087

A year of it, worked

Marcus asks one question about your number: per year, at our volume. It is one multiply, and doing it out loud is worth more than another decimal place on the number itself.

STEP	THE ARITHMETIC	COMES FROM
Disruption chats a year	about 530,000	The case study's five lines
Humans still take 42% of in-scope chats	$530,000 \times 0.58 = \text{about } 307,000$ the agent path can reach	Larkspur's week-12 containment
At \$0.14 a resolved contact	$307,000 \times \$0.14 = \text{about } \$43,000$ a year	Larkspur's loaded number at week 7
The human baseline for those same contacts	$307,000 \times \$6.90 = \text{about } \$2,118,000$ a year	The case study's five lines
The gap	about \$2,075,000 a year	Subtraction, and it is not a saving yet

Say the caveat on the same slide as the number, in three parts. The 223,000 chats humans still take cost about \$1,540,000 a year and do not go away. Everything that is not chat is out of scope, and that is 59% of contacts. And a gap is not a saving until somebody does not buy something: Larkspur counted theirs as 60 seasonal BPO (outsourced contact-center) seats they did not buy, about \$410,000, because they ran at 50% exposure for one storm season rather than at 100% for a year.

Then do it again with your own after number instead of \$0.14, and say which one you used. Yours is model cost from `bench.py` ; \$0.14 is Larkspur's loaded number. Larkspur's model cost was about 40% below its loaded number, and a client will ask which you used.

4.1 Pull the lever, measure it

WHAT YOU ARE BUILDING

Bench before you tune. The before number cannot be rebuilt after.

A win that breaks a Stage 1 shape fails, and the gate checks both sides.

DO

1. Declare your lane in `PITCH.md` and in your pod thread. Replace everything after the colon on the `Lever:` line with the one word you picked; the gate reads that word and refuses the menu.

```
Lever: cost
```

2. Write the caveat now, before anyone benches. Make it one sentence: what this lever costs, and what the number about to arrive will not cover.
3. Bench on your own laptop, labeled `before`, before any change lands. Use three runs per shape on both sides, and `--cold` so the run pays for its own cache.

```
$ python3 bench.py --label before --stage 1 --runs 3 --cold
```

4. Bench Stage 2 on both sides of the swap if you are on the intelligence lane. Your metric is the wire-rule count, and it only exists on Stage 2.

```
$ python3 bench.py --label s2-before --stage 2 --runs 3
# then, after the swap
$ python3 bench.py --label s2-after --stage 2 --runs 3
```

5. Pull one lever in the lane you declared, from the table above or one of your own that bench can read. Build it yourself or with Claude Code.
6. Bench again and diff the two labels, same stage and the same number of runs on both sides. Then read the cache line under the table.

```
$ python3 bench.py --label after --stage 1 --runs 3 --cold
$ python3 bench.py --compare before after
```

7. Run the five shapes again before you call it done.

```
$ python3 run.py --all --trace
```

8. Then run the gate.

```
$ python3 verify.py 4.1
```

9. Back in the demo, read the number at the top of the panel and the before and after rows under it. Then write the client page, open it, and screenshot both for the pitch.

```
$ python3 readout.py --client
$ open readout-client.html
```

10. One person pushes the canon at the end of the build. Agree who before the clock runs out, and everyone else takes it.

```
$ python3 readout.py
$ python3 pod_sync.py --push-canon
```

YOU WILL KNOW IT WORKED WHEN

On the cost and speed lanes, your number moved at least 10% in the right direction. On the intelligence lane, at least one more Stage 2 case satisfies its rules, and a hard-gate suite that was failing now passes on more than half its runs. Either way, Stage 1 still resolves 5/5. Ten percent is there so noise does not read as a win. A 70% cost win that breaks a shape fails. The comparable check is the one people trip over: it wants the same stage, at least three runs per shape on both sides, and the same model, except on the intelligence lane, whose lever is the model swap. The first check reads the `Lever:` line in `PITCH.md`. The numbers below are one reference run on the cost lane; yours will differ.

```
$ python3 verify.py 4.1
```

Step 4.1: Build 4 · Make it scale

- ✓ `PITCH.md`: the `'Lever:'` line names one lane (cost)
- ✓ a `'before'` bench exists
- ✓ an `'after'` bench exists
- ✓ the two runs are comparable (3 runs per shape each)
- ✓ stage 1 still resolves 5/5 after your change (100%)
- ✓ cost lane: `model_cost_per_contact` 0.0564 → 0.0233 (-59%)

All checks passed. Evidence code: 61A-096 (banked for "Alice Example")

That code is your receipt for step 4.1. Paste it into the build site to bank it. Nothing to upload.

<https://virtual.partnerbasecamp.com/build/>

IF NOT
YOU SEE

WHERE TO LOOK

The number went the wrong way

Read the bench file before you touch the agent again: cache read and written, tokens, turns.

The bench says `cache: not used on any turn`

Read what the request marks for reuse. If nothing is marked, there is no cache at all, and that is not the same as a 0% hit rate.

Caching reports a 0% hit rate

Print what goes out at the top of every request, two calls in a row, and find which characters are not identical.

Cost fell and latency rose

Check how many runs per shape are behind both numbers. If the latency rise is still there at three runs per shape, it is a real charge, and it belongs in the pitch.

The bench says the run read a cache it did not pay to write

Re-bench with `--cold`, or put the word warm next to the number in the pitch. An earlier run already paid for that cache.

The gate says the two runs are not comparable

Read the stage, the runs per shape, and the model the gate prints for each side. It will not compare below three runs per shape, and it holds the model still unless your lane is intelligence.

IN CLAUDE CODE

Type `claude` in the repo folder, then `/build`. Do this after the before bench, while you are deciding which lever to pull.

Read `.workshop/bench-before.json` and my latest trace, and tell me where the tokens go per turn. Ask me which lever I think that points to, and why, before you say yours. Do not change `agent.py`.

IF YOU ARE AHEAD · after the gate, never instead of it

Try a second lever. Leave `before` and `after` untouched, they are your banked pair. Pull a second lever, bench it as `after2`, and compare `after` against `after2`. The payoff is not the row that improved. It is the row that got worse, with its percentage. That is the Charges column, on your own build.

Name the row that got worse, with its number, in `PITCH.md`.

NEXT

The value conversation. Somebody who owns a budget is about to ask you what a year of this costs.

Provable now, and not yet

This build is where a number exists for the first time, so it is where the temptation to round up lives. A pod that names the loaded-cost gap before a client finds it is the pod that gets asked back.

PROVABLE NOW	NOT YET
Your declared lane's number moved, before and after, on your own laptop, 3 runs per shape.	You cannot prove the loaded number yet. <code>bench.py</code> reports model cost, and Larkspur's model cost was about 40% below its loaded number.
Stage 1 still resolves 5/5 after the change, so the win did not break a shape.	You cannot prove accuracy at volume yet, or anything on real traffic.
You can prove what the lever charged you, if you benched 3 runs per shape on both sides. The reference cost lever read a fifth slower at 1 run each and no slower at 3, which is the noise floor on this repo's own headline number.	You cannot prove yet that the same lever holds on a storm day, at concurrency, against a slow backend.

The value conversation

Nobody in the audience is neutral. Whoever is not pitching is the client, and clients own budgets. You are paired across lanes, so the client opposite you moved a different number and knows exactly where your claim is weakest.

Your role play, in 25 minutes

You pitch once. You sit in the client's chair once.

PHASE	MIN	WHAT HAPPENS
You pitch	5	Show the demo, state the claim, name the caveat
They are the client	4	Ask two questions from the four axes, not four soft ones.
Swap, they pitch	5	Same three beats
You are the client	4	Two questions back
Everyone writes	3	Your three lines, filed as an issue on the repo of the pod you just heard
Regroup	4	Back in the main room once your issue is filed

Your five minutes

- Open with the demo running. A demo that works earns more trust in the first thirty seconds than any number you say after it.
- State the claim: one number, its unit, and what it is per.
- Give the two numbers, before and after, from your own bench files.
- Name the caveat, the thing you did not measure.
- Then stop. The four minutes after this are theirs.

5 MINUTES ON THE CLOCK

What you bring

- **1 claim:** a number, its unit and its denominator, on the one metric your pod declared.
- **2 numbers:** before and after, straight from your own two bench files.
- **1 caveat:** what you did not measure, named by you before she finds it.

The caveat is scored, so a pod that hides a gap loses more than a pod that owns one.

The client team, and where they push

Pick one of them before the pitch starts, and ask two questions from the axes. In the Sounds like row, "whose name is on it" means who is accountable for it.

THE AXIS	PRIYA · PROJECT MANAGER	MARCUS · HEAD OF OPS
Sounds like	"Which model should we be on, and how will you prove it?"	"What does a year of this cost, and whose name is on it?"
cost	Per resolved contact	Per year, at our volume
wrong	The first untrue thing it says	Who is accountable when it is
operate	Who runs it in June	The day your team leaves
scope	What you left out, and why	Where the kill switch lives

- If your number is cost or speed, take Marcus. He is harder to convince.
- Your questions are graded too. Four polite questions and a compliment is how this block gets wasted.

The third chair

Priya and Marcus both want this to work. The third person in the room does not, yet, and they can end it in one meeting.

Ask all 3 when you are the client, and have all 3 answered when you are not.

THE QUESTION	WHERE YOUR ANSWER COMES FROM
Whose data does it touch?	Look in Appendix D's reads table. <code>get_flight_status</code> is the one read that is not fenced to the customer's own booking, and you name it yourself.
What happens after the first untrue answer?	Say which gate catches it and what the customer sees next. <code>send_confirmation</code> is the only write the customer reads that has no threshold and no token.
Who signs a policy change?	A policy change is a row edit with a changelog line, against a named owner and a version. A prompt release is not a policy change.

Everyone writes

THE LINE	WHAT IT IS
A number you would take to your own client	It comes from their run, not from their slide. It is one you would repeat with your own name on it.

A NUMBER YOU WOULD TAKE TO YOUR OWN CLIENT

A claim you would not sign

It is the one their evidence does not carry yet.

A CLAIM YOU WOULD NOT SIGN

A gap

It is the thing you would want closed before this goes near a customer.

A GAP

GOES TO an issue on the other pod's repo

File the three lines as an issue on the other pod's repo, then read the one filed on yours before the close.

Before you close the laptop: ACCOUNT.md

`ACCOUNT.md` holds three lines, in your repo at the root, and nobody can write them for you. Fill them here, copy them into `ACCOUNT.md`, and commit the file with the readout.

THE LINE

WHAT IT IS

Account

The account you take this to

Name one company, not a segment or a vertical.

ACCOUNT

Workflow

The job it would do there

Pick one job you understand end to end: who starts it, what it reads, what it may change, what stays human.

WORKFLOW

Date	When you have the conversation	Pick a date inside 30 days that is already in somebody's diary, or about to be.
------	--------------------------------	---

DATE

GOES TO ACCOUNT.md in your repo

Are the three lines in ACCOUNT.md and committed? That is the two sessions, done.

A · Working in one repo

What is in the repo

WHERE	WHAT IT IS
<code>guide/</code>	It holds this document. Open <code>index.html</code> , or read the PDF on paper.
<code>setup.py</code>	Checks this machine. Run it until the last line says READY.
<code>agent.py</code>	The file you edit has about 200 lines: the loop and three empty slots on top, and below the fold, what Claude is told about each of the nine tools.
<code>run.py</code>	Runs the agent on one ticket and shows every turn on the wire. It is the command you type most.
<code>verify.py</code>	It holds the gates. You run one by step id.
<code>support/</code>	It holds the nine tools' real functions. They are given, correct, and never edited.
<code>bench.py</code> · <code>readout.py</code>	Measure the agent, and write the page that shows what it did.
<code>eval_harness.py</code>	Runs your pod's own eval cases against your agent.
<code>demo/</code>	It is the demo: chat on the left, evidence on the right. It is given, and it runs as shipped.
<code>data/americas/</code>	It holds bookings, flights, and the policy engine.
<code>pod_sync.py</code>	It handles the one <code>agent.py</code> your pod agreed on: push it, take it, or <code>--status</code> .
<code>evals/</code>	It holds worked examples now, and your own cases later.
<code>PITCH.md</code>	It holds your pod's six lines, and the 4 answers to Priya your pod writes at the open of session two.
<code>ACCOUNT.md</code>	It holds the account you take this to, its workflow, and a date inside 30 days: 3 lines, filled at the close.
<code>TEAM.md</code>	It holds your pod's name and roster, typed once by whoever created the repo. The review reads the names.

`build2_probe.txt`

It holds your pod's one test message for a tool you write yourself. You create this one.

`readout.html` · `readout-trace.json`

They are written at the repo root by `readout.py`, and committed when the canon is published.

`readout-client.html`

It holds the same evidence in the client's words, from `python3 readout.py --client`. It is one page you can send.

The whole path

Five commands carry the build, and the repo's README carries the same five. The rest are named on the step that needs them, so there is nothing else to hold in your head now.

In this repo, `claude` gives you `/setup`, `/build`, `/check` and `/readout`. `/build` builds the step with you and never hands you the answer, and saying build the agent in plain words does the same, and `/check` runs the gate and translates what it printed, which is the shortest way in if you would rather not lead with a terminal.

Two more appear at the close, and neither is on the clock: `python3 readout.py --client` writes the page you can send a client, and `ACCOUNT.md` is 3 lines you type by hand.

```
$ git clone <your pod repo URL>      # once
$ python3 setup.py                    # until it says READY
$ python3 run.py K7PQ2M --trace       # every build
$ python3 verify.py 1.2               # every gate
$ python3 pod_sync.py --take-canon    # at the end of a build
```

Here is the rule, in one sentence. **Nobody commits `agent.py` mid-build. At the end, the build's committer publishes the canon and everyone else takes it.**

```
$ python3 pod_sync.py --push-canon    # the committer
$ python3 pod_sync.py --take-canon    # everyone else
```

Everyone builds their own `agent.py` on their own laptop. Several people editing one file in one repo at the same time produces a merge conflict inside a loop they are all still learning to read, and no build has time for that. So the repo only ever holds one version: the one the pod chose.

`--take-canon` saves your version into `.workshop/mine/` first and prints the path. Nothing you wrote is lost, and diffing yours against the canon is the most useful five minutes of the build.

`PITCH.md` and `evals/cases.json` are the pod's shared record and they are normal commits. The bench pair never leaves your laptop.

Who publishes the canon

One person pushes the canon at the end of the build. Agree who before the clock runs out. Everyone else takes it.

Two canon pushes in one build is the merge conflict this whole protocol exists to prevent, so it has to be one person, and the pod has to have agreed who before the clock runs out.

Behind, or something broke

The pod repo is the checkpoint. This restores what the pod pushed.

```
$ python3 pod_sync.py --take-canon --force
```

What it does not restore

If `.workshop/` is already gone, say so rather than re-benching `before` against the current agent. That number would be a fiction, and your pod's claim rests on it.

NOT RESTORED	WHAT TO DO
<code>.env</code> , your key	It is yours, and never in the repo. Re-create it from <code>.env.example</code> .
<code>.venv/</code>	Rebuild it with <code>python3 setup.py --fix</code> .
<code>.workshop/</code> , your banked gates and bench numbers	It is not on the remote. Copy it across before you re-clone into a fresh folder, and copy it back in after.

B · The five tickets

The table shows one example of the bar, not the only wording that passes. Two of the five are not what they look like: one should be handed to a person, and one gets an answer this build has no gate for.

PNR	SHAPE	WHAT GOOD LOOKS LIKE
K7PQ2M	Clean cancellation	Holds a seat, then waits for the customer's click
M3XR8T	Delay under threshold	Refuses the waiver and names the policy row it came from
T9WN4C	Ambiguous missed connection	Asks a clarifying question before it does anything
G2HL9V	Out of scope, a group of 12	Escalates. It hands the customer to a person, and that is the pass
R8KD3F	Abuse plus a legal threat	Comes back calm and helpful. There is no tone gate yet

C · The words of the build

WORD	WHAT IT MEANS HERE
PNR	It is the booking reference, six characters. It is how every tool finds this customer.
The wire	It means the requests and replies that went to Claude and came back. <code>-- trace</code> prints it, and every gate reads it.
Turn	It means one request plus one reply. Nothing carries over on its own: you send the whole transcript again, and that is all the context Claude has.
<code>stop_reason</code>	It is the field that ends a turn or continues it. <code>tool_use</code> means Claude is waiting on an answer, <code>end_turn</code> means it is done.
<code>tool_result</code>	It is the block you send back with what a tool returned, keyed to the id of the request that asked for it.
Slot	It is one of the three empty lines at the top of <code>agent.py</code> , each labelled with the build and step that fills it. Each is empty from the first minute, so nothing new ever arrives in the file.
The fold	It is the ruled line partway down <code>agent.py</code> . Above it sit the loop and the slots. Below it is what Claude is told about each of the nine tools.
Schema	It is the name, the input fields and the description Claude reads for one tool, not the function that answers it.
Tool description	It is the only thing Claude reads to pick a tool: a briefing, not docs.
Shape	It is one kind of customer situation. The five tickets are five shapes.
Stage 1	It means the five tickets in the case study. Every build has to still resolve all five.
Stage 2	It is the harder set: five adversarial messages. Only the intelligence lane benches it.
Gate	It is a check your agent passes before the build moves on. <code>python3 verify.py <step></code> runs one.
Hard gate	One failure blocks the release, never an average.

Evidence code	It is what a passed gate prints on its last line: your receipt, made from your name, one per person. Paste it into the build site.
Canon	It is the one <code>agent.py</code> the pod agreed on, published at the end of a build. Until you take it, what is on your laptop may not be the canon.
Probe	It is your pod's one test message for a tool you wrote, three lines in <code>build2_probe.txt</code> : PNR, last name, the customer's message.
Readout	It is the one page your repo writes about what your agent is and what it just did, from <code>python3 readout.py</code> .
Bench	It is one timed run of the agent over a fixed set of tickets, labeled so two runs can be diffed. <code>bench.py</code> calls one of these a sweep.
Lane	It is which number you are moving in Build 4: cost, speed or intelligence. The <code>Lever:</code> line in <code>PITCH.md</code> names it.
Lever	It is the one change you make to move that number.
Prefix	It is the front of a request, the part that does not change between calls. It is the part caching can reuse.
p50	It is the response time half of contacts come in under: the middle one, and the number the speed lane is graded on.
p95	It is the response time 95 of every 100 contacts come in under. It is the number an SLA is written against, not the average. At the sample sizes a build reaches it is literally the slowest conversation in the run, which is why <code>bench.py</code> prints it as slowest observed and the speed lane is graded on p50.

D · What this agent cannot do, and what stops it

This is the page to hand a CIO. It answers the two things they ask first, whose data it touches and what stops it doing something nobody authorized, and it answers them from the code rather than from a slide.

Every line here is sourced from `support/tools.py` and `data/americas/disruption_policy.json`. This is the map of what is fenced, so nothing is written into it that the code does not do.

`python3 readout.py --client` puts these same two tables on a one-page HTML file you can send, with no retyping.

Reads

TOOL	WHAT IT DOES
<code>lookup_booking</code>	Takes a PNR and a last name, and refuses if the name is not on that booking. Returns a trimmed view, never the raw record, and labels the ops note and the remarks as untrusted free text.
<code>get_flight_status</code>	Takes a flight number and a date. It is the one read not fenced to the customer's own booking, so it is the one that can be pointed anywhere.
<code>search_alternatives</code>	Takes the PNR alone, so it can never be pointed at a route the customer did not buy.
<code>check_policy</code>	Re-derives fare family, loyalty tier and whether this is an overnight from the booking on every call, so a model cannot talk its way into an entitlement the data does not support.

Writes

TOOL	WHAT IT DOES	THE FENCE
<code>hold_seat</code>	Puts a seat aside and hands back a hold that expires in 15 minutes.	It is reversible, and it undoes itself if nobody confirms.
<code>issue_voucher</code>	Issues on its own only under the policy threshold: 25 dollars for a meal, 40 for ground, 75 for goodwill.	A hotel is never automatic, and anything over the line queues for a human.

<code>confirm_rebooking</code>	Reissues the ticket. It is the irreversible one.	It runs only with the customer's own click token. "The customer said yes" in chat is not it.
<code>send_confirmation</code>	Writes the message the customer reads.	It is the only write that reaches the customer with no threshold, no token and no undo, so it carries whatever the agent got wrong.
<code>escalate_to_human</code>	Files the case with a written summary for whoever picks it up.	The correct outcome for groups, refunds, and anything the policy does not cover.

BASECAMP

Prepared for Partner Basecamp participants. Not for reproduction or redistribution as training material. You are free to apply these patterns in your own client work. Larkspur Airlines is fictional; data/americas/ is the authority for every fact the agent states.